

Unit testing with JUnit

CSC103 Programming Fundamentals / Lecture 31

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Write a Jupiter test asserting that a mark validator rejects 101 and accepts 100.

Use the same validator contract as the production method.

Learning objectives

- Write a focused unit test
- Use equality and exception assertions
- Separate test setup, action and checks

CLO-4

A unit test

JUnit structure

Assertions for behaviour

Setup, action and checks

A unit test

- A unit test checks a small unit of behaviour, usually through a method.
- A test name explains the case and expected outcome.
- Keep the test independent of other tests.

Example

Test name: `largerOfTwoNegatives`

Given: -4 and -2

Expected: -2

JUnit structure

- @Test marks a Jupiter test method.
- Assertions fail the test when a result violates the expectation.
- Production code and test code belong in separate source folders.

Example

```
@Test
void largerOfTwoNegatives() {
    assertEquals(-2, Numbers.max(-4, -2));
}
```

Assertions for behaviour

- assertEquals compares an expected result to the actual result.
- assertTrue checks a condition. assertThrows checks an exception contract.
- For approximate doubles, choose a justified tolerance.

Example

```
assertEquals(2.5, mean(2,3), 1e-9);  
assertThrows(IllegalArgumentException.class,  
    () -> validateMark(-1));
```

Setup, action and checks

- Arrange the input and required state.
- Act by calling the target.
- Assert the outcome without depending on a previous test.

Example

```
Arrange: int[] a = {2,3};  
Act: int[] b = doubled(a);  
Assert: b is {4,6}; a is still {2,3}.
```

Exception assertions execute an action

- `assertThrows` receives an expected exception type and an action to execute.
- The action must be deferred so the assertion can observe its failure.
- A lambda supplies that action without running it during argument preparation.

Example

```
assertThrows(IllegalArgumentException.class,  
    () -> validateMark(101));
```

A passing boundary test checks inclusion

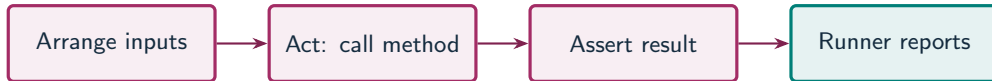
- The upper allowed value, 100, should return normally.
- The next value, 101, should throw the documented exception.
- Testing both distinguishes inclusive and exclusive range implementations.

Example

Valid condition: `0 <= mark && mark <= 100`

Reject condition: `mark < 0 || mark > 100`

A unit test separates setup, action and check



What to notice

A passing assertion is evidence about its chosen case, not proof for every input.

Key distinctions

Assertion	Checks	Typical use
assertEquals	Expected and actual values	A numerical result
assertArrayEquals	Element contents	An array transformation
assertThrows	Expected exception type	Invalid input
assertDoesNotThrow	Normal completion	A valid boundary input

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static int max(int a, int b) { return a >= b ? a : b; }
```

First example: execution

```
int result = max(-4, -2);  
if (result != -2) throw new AssertionError("negative pair");  
System.out.println("Unit behaviour passed");
```

First example: result

Unit behaviour passed

The companion JUnit project expresses this same check with `@Test` and `assertEquals`.

This console demonstration has no external dependency.

Assertions for behaviour

Setup, action and checks

Guided practice

Write a Jupiter test asserting that a mark validator rejects 101 and accepts 100.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Use the same validator contract as the production method.
- Assert that 101 throws `IllegalArgumentException`.
- Assert that 100 completes without throwing.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution31 {  
    public static void main(String[] args) throws Exception {  
        org.junit.jupiter.api.Assertions.assertThrows(  
            IllegalArgumentException.class, () -> validateMark(101));  
        org.junit.jupiter.api.Assertions.assertDoesNotThrow(  
            () -> validateMark(100));  
        System.out.println("2 checks passed");  
    }  
    static void validateMark(int mark) {  
        if (mark < 0 || mark > 100) {  
            throw new IllegalArgumentException("mark outside 0..100");  
        }  
    }  
}
```

Execution trace

Input	Expected behaviour	Assertion	Result
101	Throws IllegalArgumentException	assertThrows	Pass
100	Returns normally	assertDoesNotThrow	Pass

Follow the changing state in execution order.

Solution result

2 checks passed

Why this result follows
Assert that 100 completes without
throwing.

A common incorrect approach

```
@Test void test() {  
    Numbers.max(2,5);  
}
```

Example

The test checks no outcome. Add `assertEquals(5, Numbers.max(2,5))` so an incorrect `return` value fails the test.

Boundary and failure checks

Check both endpoints 0 and 100, then -1 and 101.

Use `assertThrows` for mark 101 and `assertDoesNotThrow` for mark 100. Put each case in a separately named test; the complete test class follows.

A complete Jupiter test class (1/2)

```
package edu.cui.csc103;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class BoundaryTest {
    @Test
    void rejectsMarkAbove100() {
        assertThrows(IllegalArgumentException.class,
```

A complete Jupiter test class (2/2)

```
        () -> Numbers.validateMark(101));  
    }  
  
    @Test  
    void acceptsMarkAt100() {  
        assertDoesNotThrow(() -> Numbers.validateMark(100));  
    }  
}
```

Check your understanding

Does an assertion-free test prove a returned value is correct? Why use `assertArrayEquals`?

Write a prediction and explain the rule that supports it.

Answer and explanation

No. It may only establish that the call returned without an uncaught failure. `assertArrayEquals` checks element contents rather than array identity.

The test checks no outcome. Add `assertEquals(5, Numbers.max(2,5))` so an incorrect return value fails the test.

Compare cases and predict outcomes

Test input	Expected behaviour	JUnit assertion
mark 100	Accepted	<code>assertDoesNotThrow</code>
mark 101	Rejected	<code>assertThrows</code>
mean of 40 and 60	50.0	<code>assertEquals</code> with tolerance

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write a Jupiter test for `mean({40,60})` using the supplied `Numbers` class. Identify the expected value and tolerance.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
@org.junit.jupiter.api.Test
void meanOfTwoMarks() {
    double actual = Numbers.mean(new int[]{40, 60});
    org.junit.jupiter.api.Assertions.assertEquals(50.0, actual, 1e-9);
}
```

- The independently calculated expected mean is 50.0.
- The delta 1e-9 is the allowed absolute difference for this example.
- Place the method inside a test class in the companion project.

Lecture summary

- a focused unit test
 - equality and exception assertions
 - Separate test setup, action and checks
- Use the same validator contract as the production method.
 - Assert that 101 throws `IllegalArgumentException`.
 - Assert that 100 completes without throwing.

After-class practice

Add JUnit tests for negative values, ties and invalid input to the supplied Numbers example.

Syllabus reading

Reference material: JUnit Jupiter

Next lecture: Testing practice and course synthesis